Functions and Exception Handling in C#.Net



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



- Introduction to Functions
- Return Value from Function
- ByVal and ByRef Parameters
- Out Paramater
- Optional Paramaters
- Named Parameters
- Pass an array as an argument using params
- Versions of main functions in C#
- Function Overloading
- Recursive Functions
- Expression-Bodied Member
- Introduction to Exception Handling in C#
- Exception and Exception Handling
- How to create User Defined Exception

Introduction to Functions

In C#, functions are a fundamental building block of code that encapsulate a specific set of instructions and can be invoked to perform a specific task or return a value.

- Functions provides several benefits:
  - **Modularity and Reusability:** Functions allow you to break down complex tasks into smaller, more manageable units of code. This modular approach makes the code easier to understand, maintain, and reuse. Functions can be called from multiple places within a program or even in different programs, promoting code reuse and reducing duplication.
  - **Encapsulation:** Functions encapsulate a set of instructions, data, and algorithms, providing a clear and well-defined interface. Encapsulation helps in hiding the internal implementation details of a function, making the code more maintainable and promoting separation of concerns.

- **Code Organization:** Functions help in organizing code by grouping related operations together. This improves code readability and maintainability. By giving meaningful names to functions, you can also enhance the understandability of the code.
- **Abstraction and Abstraction Levels:** Functions allow you to create abstractions, which provide a higher-level view of a complex task or process. By encapsulating implementation details, functions provide a simpler interface for interacting with the underlying logic. This promotes code abstraction, making the code more concise and easier to comprehend.

- **Code Reusability and Maintainability:** Functions promote code reusability as they can be called multiple times from different parts of the program or even in different programs. Reusing functions saves development time and effort, and it helps ensure consistency and reduce bugs.

- **Testing and Debugging:** Functions facilitate unit testing, as you can isolate and test individual functions independently. This helps in identifying and fixing issues more effectively. Functions with well-defined input and output parameters are easier to test and debug, as their behavior can be validated independently.

- **Readability and Understandability:** Functions improve code readability by encapsulating specific tasks and providing a higher-level view of the program's logic. Well-structured functions with meaningful names and a clear purpose make the code easier to understand, especially for other developers who may work on the codebase.

Syntax:

```
access-modifier return-type FunctionName(parameters)
{
    // Function body - contains the code to be executed
    // when the function is called
    // It may include variable declarations, statements,
and expressions

    // Optional: return a value using the return statement
    return value;
}
```

Example:

```
public int CalculateSquare(int number)
{
    int square = number * number;
    return square;
}
```

```
int result = CalculateSquare(5);

Console.WriteLine("Square: " + result);
```

Return Values in a C# Method

In a C# function you may want to return the result of an operation. You can return the value of a variable in a function by using the return keyword.

If u place only single statement without return type it will take u out of code block. void is also return type of null.

Syntax:

```
public int AddNumbers(int a, int b)
{
    int sum = a + b;
    return sum;
}
```

```
int result = AddNumbers(3, 5);
Console.WriteLine("Sum: " + result);
```

Call ByVal and Call ByRef Parameter

In C#, parameters can be passed to functions using two different methods: "call by value" and "call by reference". Let's understand what each of these methods means:

Call ByVal	Call ByRef
When a parameter is passed by value, a copy of the parameter's value is created and passed to the function.	When a parameter is passed by reference, a reference to the original parameter is passed to the function.
Any modifications made to the parameter within the function do not affect the original value outside the function.	Any modifications made to the parameter within the function affect the original value outside the function.
Primitive data types (such as int, float, bool) and structs are typically passed by value.	Reference types (such as objects, arrays, and classes) are typically passed by reference.
No keyword is required.Its by default	The ref keyword is used when you want to pass a parameter by reference and require it to be initialized before passing.

```
void ModifyValue(int num)
{
    num = 10;
    Console.WriteLine("Modified value inside function: " + num);
}

int value = 5;
ModifyValue(value);
Console.WriteLine("Original value: " + value);
// Output:
// Modified value inside function: 10
// Original value: 5
```

```
void ModifyValue(ref int num)
{
    num = 10;
    Console.WriteLine("Modified value inside function: " + num);
}

int value = 5;
ModifyValue(ref value);
Console.WriteLine("Modified value: " + value);
// Output:
// Modified value inside function: 10
// Modified value: 10
```

The C# out Parameters

In C#, the out keyword is used to define an output parameter in a method or function. An output parameter is similar to a reference parameter (ref), but with one key difference: it allows the method to assign a value to the parameter even if it was not initialized before calling the method.

Example:

```
bool TryDivide(int dividend, int divisor, out int
quotient)
{
    if (divisor != 0)
    {
        quotient = dividend / divisor;
        return true;
    }
    else
    {
        quotient = 0;
        return false;
    }
}
```

```
int dividend = 10;
int divisor = 0;
int resultQuotient;

bool isDivisionSuccessful = TryDivide(dividend, divisor, out
resultQuotient);

if (isDivisionSuccessful)
{
    Console.WriteLine("Division successful. Quotient: " +
resultQuotient);
}
else
{
    Console.WriteLine("Division failed. Cannot divide by zero.");
}
```

The Optional Parameters

By default, all parameters of a method are required. But in C# 4.0, the concept of optional parameters was introduced that allows developers to declare parameters as optional.

That means, if these arguments are not passed, they will be ommitted from the execution. Optional parameters are not mandatory.

Example:

```
public int Size(int Height, int Width=1)
      {
         return Height * Width;
      }
```

```
static void Main()
      {
         MyMath m = new MyMath();
        Console.WriteLine( m.Size(80));
         Console.ReadLine();
      }
```

Named Parameter

Named Parameters allow developers to pass a method arguments with parameter names. Its useful while using multiple optional parameter.

Example:

```
public string Size(int Height, int Width=1,string msg="The Size is: ")
        {
            return msg + Height * Width;
        }
```

```
static void Main()
        {
            MyMath m = new MyMath();
          Console.WriteLine(   m.Size(80,msg:"The Answer is"));
            Console.ReadLine();
        }
```

How to pass array to a function as a parameter

- **Traditional Way:**

```
public void Marks(int[] marks)
      {
         foreach (int temp in marks)
         {
            Console.WriteLine(temp);
         }

      }
```

```
static void Main()
      {
         MyMath m = new MyMath();
         int[] mymarks = {50,60,70,80 };
         m.Marks(mymarks);
         Console.ReadLine();
      }
```

- **Using params keyword:**

```
public void Marks(params int[] marks)
      {
         foreach (int temp in marks)
         {
            Console.WriteLine(temp);
         }
      }
```

```
static void Main()
      {
         MyMath m = new MyMath();
         m.Marks(50, 60, 70, 80);
         Console.ReadLine();
      }
```

Scope of the Variables

C# Scope of the variables:

In C#, variable scope refers to the portion of the code where a variable is accessible and can be referenced. The scope of a variable determines its visibility and lifetime within the program.

In C#, variables can have different levels of scope:

- **Block Scope:** Variables declared within a block, such as within a set of curly braces {}, are only accessible within that block. They are typically referred to as local variables. Once the block is exited, the local variables go out of scope and cannot be accessed.

Example:

```
void MyMethod()
{
    int x = 10; // Local variable with fun scope
    if (x > 5)
    {
        int y = 20; // Local variable with block scope
        Console.WriteLine(x + y);
    }
    // x is still accessible here
    // y is out of scope and cannot be accessed here
}
```

- **Method/Function Scope:** Variables declared within a method or function are accessible throughout the entire method. They are local to the method and cannot be accessed outside of it.

```
void MyMethod()
{
    int x = 10; // Local variable with method scope
    Console.WriteLine(x);
    // x is accessible here
}

void AnotherMethod()
{
    // x is not accessible here, as it is local to
MyMethod()
}
```

- **Class Scope:** Variables declared at the class level, outside of any method, are accessible within all methods and members of the class. They are typically referred to as fields or class-level variables.

Example:

```
class MyClass
{
    int x; // Field with class scope

    void MyMethod()
    {
        Console.WriteLine(x);
        // x is accessible here
    }

    void AnotherMethod()
    {
        Console.WriteLine(x);
        // x is accessible here
    }
}
```

- **Global Scope:** Variables declared outside of any class, typically at the namespace level, have global scope and can be accessed from anywhere within the program. However, it is generally recommended to avoid global variables as they can lead to code complexity and potential issues.

Example:

```
namespace MyNamespace
{
    int globalVar = 100; // Global variable

    class MyClass
    {
        void MyMethod()
        {
            Console.WriteLine(globalVar);
            // globalVar is accessible here
        }
    }
}
```

C# Main() Function Versions

There are four versions of the Main() function:

- **static void Main()** - accepts no parameters and returns no values.
- **static void Main(string[] args)** - returns no parameters and accepts a string
  array named args.
- **static int Main()** - returns an *int* value and accepts to parameters.
- **static int Main(string[] args)** - returns an *int* and accepts a string array named args.

```
using System;

class Program
{
    static void Main(string[] args)
    {
        if (args.Length > 0)
        {
            string inputString = args[0];
            Console.WriteLine("Input string: " +
inputString);
        }
        else
        {
            Console.WriteLine("No input string provided.");
        }
    }
}
```

C# Function Overloading

Overloading is the idea of having multiple functions that are the same name. Each function must accept different types or amount of types. This is how the compiler differentiates from overloaded functions.

```
class ShapeCalculator
{
    public double CalculateArea(double radius)
    {
        return Math.PI * radius * radius;
    }

    public double CalculateArea(double length, double width)
    {
        return length * width;
    }

    public double CalculateArea(double sideA, double sideB, double sideC)
    {
        // Using Heron's formula to calculate the area of a triangle
        double semiperimeter = (sideA + sideB + sideC) / 2;
        return Math.Sqrt(semiperimeter * (semiperimeter - sideA) * (semiperimeter - sideB) *
            (semiperimeter - sideC));
    }
}
```

C# Recursion

C# recursion is the idea of a function calling itself. This may seem weird but this is a good feature to produce some elegant results. Recursion is common in advanced math and science where complicated calculations require recursion. We will provide a simple and easy example of C# recursion.

Example:

```
static int Factorial(int n)
{
    // Base case: factorial of 0 or 1 is 1
    if (n == 0 || n == 1)
        return 1;

    // Recursive call: factorial of n is n multiplied by
factorial of (n-1)
    return n * Factorial(n - 1);
}
```

```
int number = 5;
int result = Factorial(number);
Console.WriteLine($"Factorial of {number} is: {result}");
```

C# Expression-Bodied Member

In C#, a bodied expression, also known as an expression-bodied member, is a concise syntax for defining methods, properties, indexers, and other members using a **single expression**.

- It provides a more compact and readable way to define simple members that consist of a single expression.

```
class Rectangle
{
    public int Width { get; set; }
    public int Height { get; set; }

    // Expression-bodied method
    public int CalculateArea() => Width * Height;
}
```

Advantages:

- **Conciseness:** Expression-bodied members make your code shorter and more concise, especially for simple operations. This can lead to improved readability and easier maintenance.
- **Clarity:** For simple members, using an expression-bodied syntax can make the intent of the code clearer and more evident, reducing the cognitive load for readers.
- **Consistency**: Expression-bodied members help maintain a consistent coding style, making your codebase more uniform and easier for a team to understand.

- **Ease of Use**: When a method or property body is a single expression, it's often easier to write and understand, as there's less boilerplate code to manage.
- **Implicit Return:** Expression-bodied members allow for implicit return of the result, reducing the need for explicit return statements.
- **Lambda Expression Familiarity:** If you're familiar with lambda expressions, expression-bodied members can feel intuitive, as they use a similar concise syntax.

Limitations:

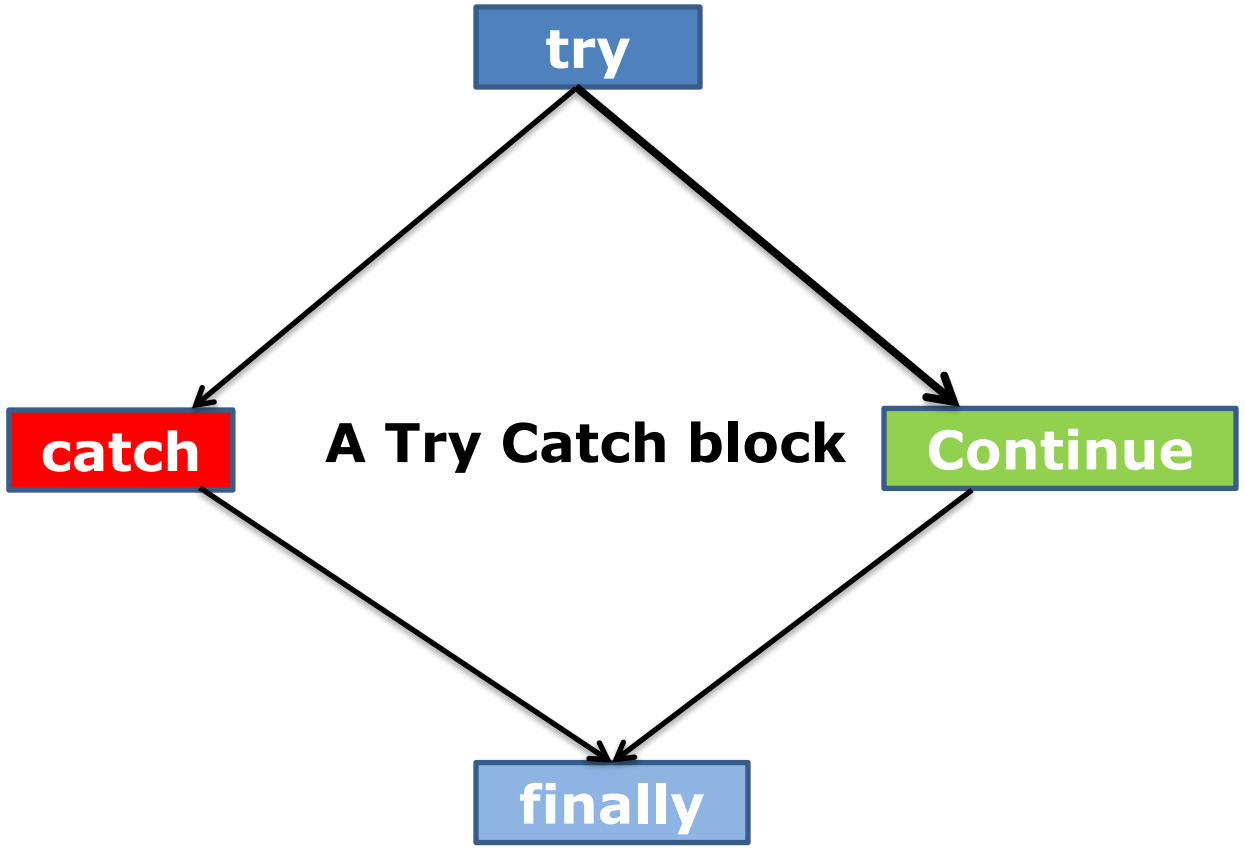
- **Complexity:** Expression-bodied members are best suited for simple operations. If your member logic becomes too complex, using a full-bodied block of code might be more appropriate for readability and maintainability.
- **Limited Debugging:** Debugging expression-bodied members can be slightly trickier compared to full-bodied members, especially if you need to inspect intermediate values within the expression.
- **Readability:** While expression-bodied members can improve readability for simple cases, they might become less readable for more complex expressions or when used excessively in a single method or class.

- **Limited to Single Expressions:** Expression-bodied members can only consist of a single expression. If you need to include multiple statements or control flow, you'll need to resort to a full-bodied block.
- **Lack of Documentation:** Expression-bodied members might require more documentation to explain their behavior, as the code itself might not be self-explanatory for more complex expressions.
- **Potential for Abuse:** Overusing expression-bodied members, especially for code that should be more expressive, could lead to code that's hard to understand and maintain.

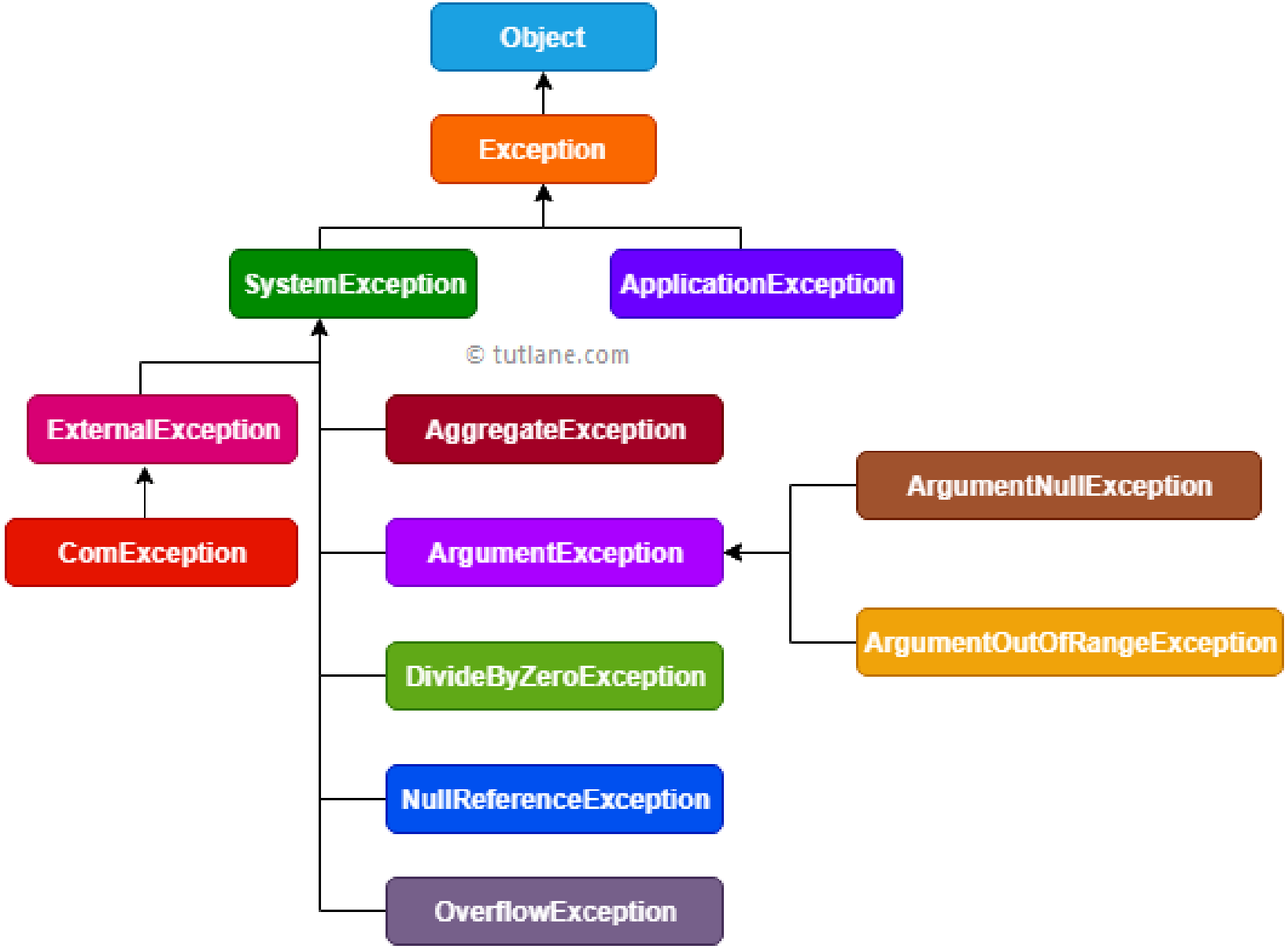
Exceptions and Exception Handling (System.Exception)

•An exception is a response to an abnormal or unexpected condition occurring in a program, due to which the execution of a program may be aborted.

•C#.NET uses a structured mechanism to handle exceptions - the Try..Catch block.



•Exception Classes Hierarchy



Example:

```
static void Main()
        {
            try
            {
                int a, b;
                a = Convert.ToInt32(Console.ReadLine());
                b = Convert.ToInt32(Console.ReadLine());
              int c = a / b;
                Console.WriteLine("Answer is :{0}", c);
            }
            catch (DivideByZeroException ex)
            {
                Console.WriteLine(ex.Message);
            }
             finally
            {
                Console.WriteLine("I am in finally");
                GC.Collect();
            }
}
```

How to use multiple catch blocks

Example:

```
static void Main()
        {
            try
            {
                int a, b;
                a = Convert.ToInt32(Console.ReadLine());
                b = Convert.ToInt32(Console.ReadLine());
              int c = a / b;
                Console.WriteLine("Answer is :{0}", c);
            }
            catch (DivideByZeroException ex)
            {
                Console.WriteLine(ex.Message);
            }
            catch (FormatException ex)
            {
                Console.WriteLine(ex.Message);
            }
            finally
            {
                Console.WriteLine("I am in finally");
                GC.Collect();
            }
}
```

Custom Exception

We have seen built-in exception classes in the previous section. However, you often like to raise an exception when the business rule of your application gets violated.

- The .Net framework includes **ApplicationException** class since .Net v1.0.

It was designed to use as a base class for the custom exception class.

- However, Microsoft now recommends **Exception** class to create a custom exception class

Example:

```
class InvalidEmployeeCode:Exception
  {
     public InvalidEmployeeCode() : base("Invalid Employee Code")
     {

     }
  }
```

```
class Employee
   {
      public void ValidateEmployeeCode(int EmployeeCode)
      {
         if (EmployeeCode <= 0)
         {
            throw new InvalidEmployeeCode();
         }
      }}
```

```
static void Main()
      {
         int code = Convert.ToInt32(Console.ReadLine());
         Employee emp = new Employee();
         try
         {
            emp.ValidateEmployeeCode(code);
         }
         catch (InvalidEmployeeCode ex)
         {
            Console.WriteLine(ex.Message);
         }
         Console.ReadLine();
      }
```

References

https://www.javatpoint.com/c-sharp-exception-handling

https://www.tutorialspoint.com/csharp/csharp_exception_handling.htm

https://www.geeksforgeeks.org/exception-handling-in-c-sharp/

https://www.c-sharpcorner.com/article/exception-handling-in-C-Sharp/

https://www.tutorialsteacher.com/csharp/csharp-exception-handling

https://www.programiz.com/csharp-programming/exception-handling

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/